

PSEUDOWEB

Manual del alumno

Compatible con la sintaxis de PSeInt · 36 ejercicios corregidos automáticamente

Versión	PseudoWeb v0.8 (sintaxis resaltada con CodeMirror)
Fecha	14 / 05 / 2026
Idioma	Español
Soporta	Linux · macOS · Windows (vía PSeInt) + cualquier navegador moderno (PseudoWeb)
Licencia	Uso educativo

Una herramienta para introducir el pensamiento algorítmico en español

Índice de contenidos

EMPEZAR DESDE CERO	Aprende a programar (para iniciarse)
Capítulo 1	¿Qué es programar? Algoritmo, pseudocódigo y diagrama de flujo
Capítulo 2	Las cinco formas del diagrama de flujo
Capítulo 3	Las once instrucciones, una a una (con ejemplos y diagramas)
Capítulo 4	Tu primer programa completo: ¿Es par o impar?
PARTE I	Sobre PSeInt y PseudoWeb
§1.1	¿Qué es PSeInt? Descarga oficial para Windows / macOS / Linux
§1.2	¿Qué es PseudoWeb? Diferencias y ventajas
§1.3	Atajos y flujo de trabajo típico
PARTE II	Referencia de sintaxis del pseudocódigo
§2.1	Estructura del programa (Proceso/FinProceso)
§2.2	Variables y tipos (Definir ... Como ...)
§2.3	Entrada / Salida (Escribir, Leer)
§2.4	Operadores aritméticos, relacionales y lógicos
§2.5	Estructuras de control (Si, Segun, Mientras, Repetir, Para)
§2.6	Arreglos (Dimension)
§2.7	Funciones built-in (matemáticas, aleatorias, cadenas)
PARTE III	Catálogo de 36 ejercicios
§3.1	Cómo se evalúa: 10 casos por ejercicio, nota sobre 10
§3.2	① Básico
§3.3	② Condicional simple
§3.4	③ Condicional múltiple
§3.5	④ Bucle Para
§3.6	⑤ Bucle Mientras
§3.7	⑥ Algoritmos clásicos
§3.8	⑦ Listas (Dimension)
§3.9	⑧ Menús con Segun
§3.10	⑨ Bucle Repetir / Hasta Que
§3.11	⑩ Números aleatorios
§3.12	■ Cadenas de texto
§3.13	■ Funciones matemáticas
PARTE IV	Apéndices
§4.1	Errores comunes y cómo solucionarlos
§4.2	Tabla resumen de constructos
PARTE V	Soluciones paso a paso de los 36 ejercicios

36 soluciones de referencia organizadas por los 12 grupos

Cada solución incluye código pseudocódigo, "Idea" y, si aplica, "Truco"

EMPEZAR DESDE CERO — Aprende a programar

Capítulo 1 — ¿Qué es programar?

Un **programa** es una lista de pasos que el ordenador sigue, uno detrás de otro, sin equivocarse y a toda velocidad. Tú lo escribes con palabras (en este caso, en español) y el ordenador lo obedece al pie de la letra.

Imagínate una receta de cocina: "1) coge los huevos, 2) cáscalos en un cuenco, 3) bátelos...". Un programa es eso, pero en lugar de manejar huevos, maneja *datos* (números, palabras, sí o no...).

¿Qué es un algoritmo?

Antes de escribir el programa, los programadores piensan el **algoritmo**: la idea general, el plan. El algoritmo no depende de ningún ordenador concreto — es la receta. El programa es esa receta escrita en un idioma que el ordenador entiende.

¿Y el pseudocódigo?

El **pseudocódigo** es un idioma a medio camino: usa palabras en español, pero con reglas fijas para que sea claro. Es perfecto para aprender, porque te concentras en el *pensamiento* sin pelearte con la sintaxis difícil de un lenguaje "de verdad" (como Python, Java o C).

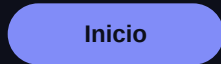
¿Y el diagrama de flujo?

El **diagrama de flujo** es el mismo programa pero dibujado. En vez de leer líneas de texto, sigues flechas que conectan formas: óvalos, rectángulos, paralelogramos y rombos. Cada forma representa una cosa distinta. En el siguiente capítulo verás las cinco formas que se usan.

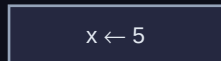
*En PseudoWeb puedes escribir tu pseudocódigo y pulsar ■ **Diagrama** para ver el diagrama de flujo correspondiente. Y al revés: si entiendes el diagrama, ya casi tienes el programa.*

Capítulo 2 — Las cinco formas del diagrama de flujo

Casi cualquier programa se dibuja combinando estas cinco formas. Apréndete su significado y serás capaz de "leer" cualquier diagrama, por complicado que parezca.



Óvalo — Marca el **INICIO** y el **FIN** del programa. Cada diagrama empieza con un óvalo "Inicio" y termina con otro "Fin".



Rectángulo — Una **operación**: una asignación, un cálculo, declarar variables, reservar un array con Dimension...



Paralelogramo — **Entrada o salida** de datos. Lo usas para **Escribir** (mostrar algo) o **Leer** (pedir un dato al usuario).



Rombo — Una **decisión**. Se evalúa una pregunta de sí/no (o de varios valores en el caso de Segun) y se sale por la flecha que corresponda.



Flecha — Indica **hacia dónde va** la ejecución. El programa siempre sigue las flechas. A veces llevan etiqueta ("Sí", "No", un valor...) para indicar bajo qué condición se toma ese camino.

*Además, los diagramas suelen llevar pequeños **puntos de unión** (círculos rellenos) donde dos o más flechas se vuelven a juntar tras una decisión. No representan ninguna instrucción, sólo marcan que la ejecución se "reunifica".*

Capítulo 3 — Las once instrucciones, una a una

A continuación tienes cada instrucción explicada con un ejemplo y, cuando aporta, su trozo de diagrama de flujo. Léelas en orden la primera vez; después puedes consultarlas sueltas.

3.1 Proceso / FinProceso

¿Para qué sirve? Es la "envoltura" de tu programa. Como cuando empiezas un cuento con "Érase una vez..." y lo cierras con "...fin", todo programa empieza con Proceso y acaba con FinProceso. El nombre que pones al lado de Proceso no afecta — sirve para que tú sepas de qué va el programa.

```
Proceso Saludo
Escribir "Hola, mundo"
FinProceso
```

En el diagrama, esto se dibuja con dos óvalos:



El programa entra por el óvalo izquierdo, ejecuta lo que hay en medio y sale por el de la derecha.

3.2 Definir ... Como tipo

¿Para qué sirve? Le dice al ordenador que vas a usar una **variable** (una "caja" con un nombre donde guardar un valor) y de qué **tipo** es la caja. Imagínate cada variable como una etiqueta con un nombre pegada a una caja.

Tipo	Qué admite	Ejemplos
Entero	Números sin decimales (positivos o negativos, también 0)	5, -3, 100, 0
Real	Números con decimales	3.14, 2.5, -0.7
Cadena	Texto (entre comillas dobles)	"Ana", "Hola!"
Logico	Sólo dos valores	Verdadero, Falso

```
Definir edad Como Entero
Definir nombre Como Cadena
Definir altura Como Real
Definir mayor_de_edad Como Logico

// Varias del mismo tipo a la vez:
Definir a, b, c Como Entero
```

En el diagrama, Definir es un rectángulo (es una operación):

Definir edad Como Entero

3.3 Asignación con <-

¿Para qué sirve? Guardar un valor dentro de una variable. La flecha <- "apunta hacia donde se guarda": `x <- 5` significa "mete 5 dentro de x".

■ **No confundas** <- (asignar) con = (comparar). `x = 5` pregunta "¿x vale 5?"; `x <- 5` ordena "guarda 5 en x".

```
edad <- 16
nombre <- "Ana"
suma <- 3 + 4 // suma vale 7
doble <- edad * 2 // doble vale 32 (si edad era 16)
```

En el diagrama: rectángulo con la asignación dentro.

`doble <- edad * 2`

3.4 Escribir

¿Para qué sirve? Muestra algo en la pantalla. Puedes mezclar texto entre comillas y nombres de variables, separados por comas.

```
Escribir "Hola"
Escribir "Tu edad es: ", edad
Escribir nombre, " tiene ", edad, " años"
```

En el diagrama: paralelogramo (forma inclinada) con lo que se va a mostrar.

Escribir nombre, edad

3.5 Leer

¿Para qué sirve? Pide al usuario que escriba un valor por el teclado, y lo guarda en una variable. Cuando el programa llega a un Leer, **se queda esperando** hasta que el usuario teclea algo y pulsa Enter.

```
Definir edad Como Entero
Escribir "¿Cuántos años tienes?"
Leer edad
Escribir "Tienes ", edad, " años"
```

Misma forma que Escribir (paralelogramo), pero con "Leer":

Leer edad

3.6 Si / SiNo / FinSi — decisión simple

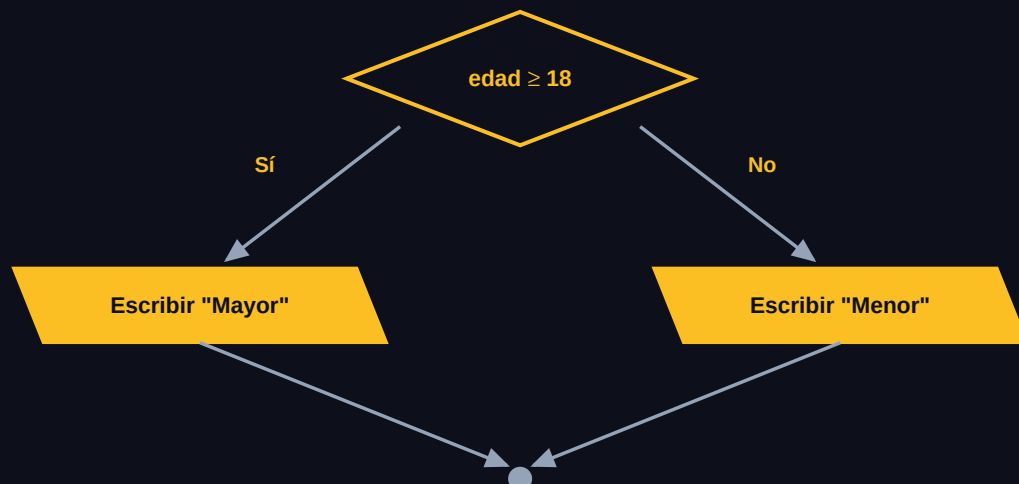
¿Para qué sirve? Permite que el programa **tome decisiones**: si una condición se cumple hace una cosa; si no, hace otra. La parte SiNo es opcional.

```
Definir edad Como Entero
Leer edad
Si edad >= 18 Entonces
  Escribir "Mayor de edad"
SiNo
  Escribir "Menor de edad"
FinSi
```

Operadores que puedes usar en la condición

Operadores de comparación	Operadores lógicos
= igual a <> distinto de < menor que > mayor que <= menor o igual >= mayor o igual	Y cond1 Y cond2 (ambas) O cond1 O cond2 (al menos una) NO NO cond (lo contrario)

Diagrama paso a paso



Cómo se lee:

- 1) El programa llega al **rombo** y evalúa la condición $\text{edad} \geq 18$.
- 2) Si es CIERTA, sigue la flecha "**Sí**" hacia la izquierda y ejecuta Escribir "Mayor".
- 3) Si es FALSA, sigue la flecha "**No**" hacia la derecha y ejecuta Escribir "Menor".
- 4) Las dos ramas se vuelven a juntar en el **punto de unión** de abajo y el programa continúa con lo que haya después del FinSi.

3.7 Segun ... Hacer / FinSegun — decisión múltiple

¿Para qué sirve? Cuando una variable puede tener varios valores y queremos hacer algo distinto para cada uno. Es un Si con muchas ramas. Más limpio que encadenar varios Si...SiNo Si...SiNo.

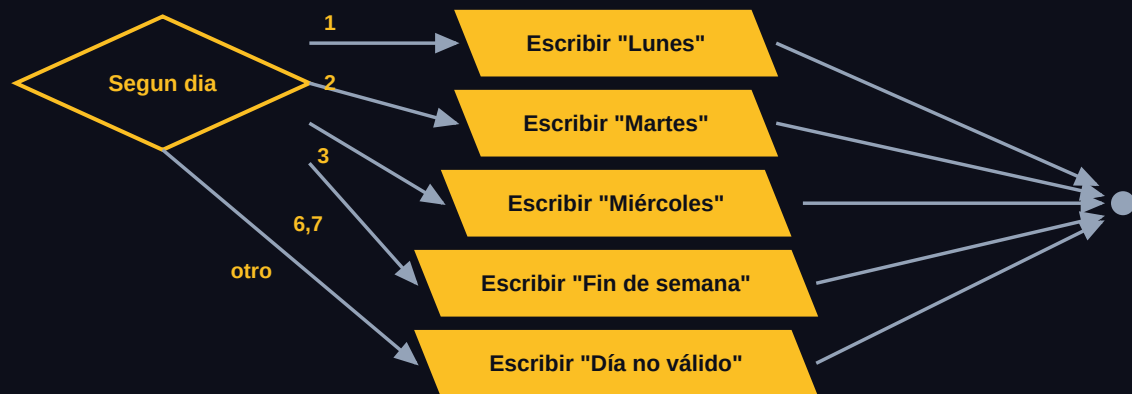
```

Definir dia Como Entero
Leer dia
Segun dia Hacer
1: Escribir "Lunes"
2: Escribir "Martes"
3: Escribir "Miércoles"
6, 7: Escribir "Fin de semana"
De Otro Modo: Escribir "Día no válido"
FinSegun

```

Cada caso lleva el valor o valores (separados por coma), dos puntos y la instrucción. De Otro Modo es opcional y se ejecuta cuando no coincide ningún caso.

Diagrama



Cómo se lee: el rombo evalúa día. Según su valor sale por una flecha (1, 2, 3, 6/7...) o por "otro" si no coincide. Todas las ramas se reúnen al final y el programa continúa.

3.8 Mientras ... Hacer / FinMientras — bucle con condición al inicio

¿Para qué sirve? Repite un bloque **mientras** se cumpla una condición. Comprueba la condición *antes* de cada repetición. Si la condición ya es falsa la primera vez, el cuerpo del bucle **no se ejecuta ni una vez**.

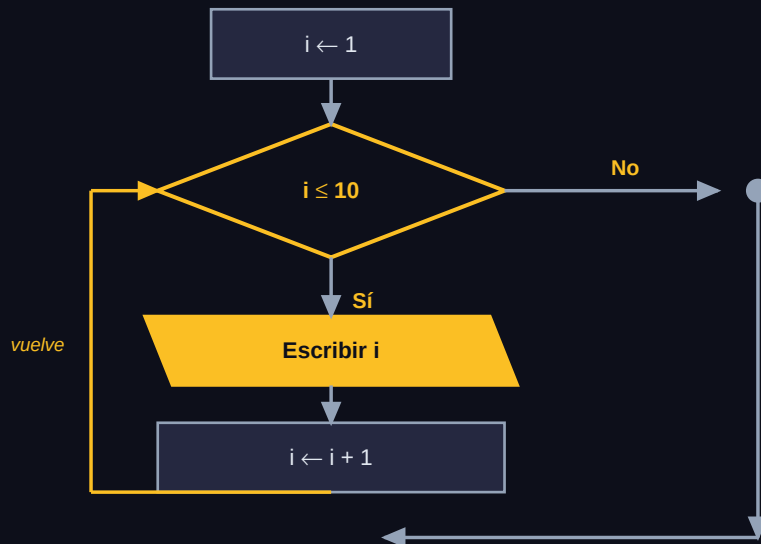
```

Definir i Como Entero
i <- 1
Mientras i <= 10 Hacer
Escribir i
i <- i + 1
FinMientras

```

■ Asegúrate de que algo cambia dentro del bucle para que la condición acabe siendo falsa. Si no, tu programa se queda repitiendo para siempre (bucle infinito).

Diagrama paso a paso



- 1) Se inicializa la variable de control: $i \leftarrow 1$.
- 2) Se llega al **rombo** y se comprueba la condición $i \leq 10$.
- 3) Si es CIERTA (Sí): se ejecuta el cuerpo del bucle (Escribir i ; $i \leftarrow i+1$).
- 4) **Al terminar el cuerpo, la flecha amarilla VUELVE al rombo** (no continúa hacia abajo). Se vuelve a comprobar.
- 5) Cuando la condición pasa a ser FALSA (No), el bucle termina y el programa sigue por la salida derecha.

3.9 Repetir ... Hasta Que — bucle con condición al final

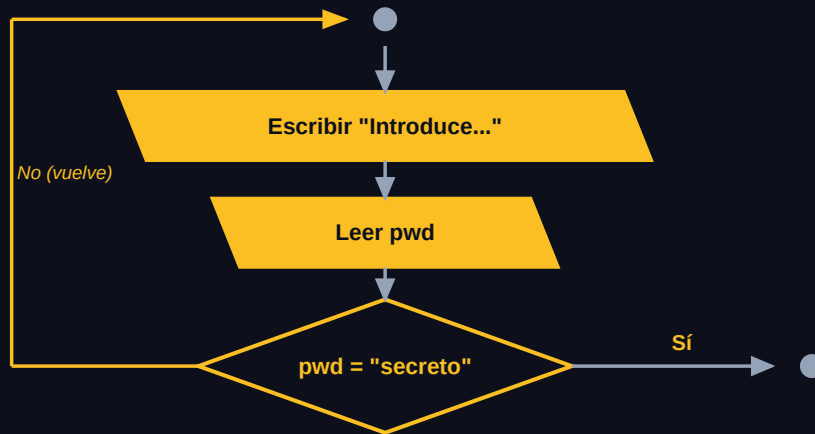
¿Para qué sirve? Como Mientras, pero comprueba la condición **al final**. Eso significa que el cuerpo **siempre** se ejecuta al menos una vez.

■ **Atención:** la condición de Repetir es de **SALIDA**. Cuando se cumple, el bucle **TERMINA** (al revés que en Mientras).

```

Definir pwd Como Cadena
Repetir
  Escribir "Introduce la contraseña:"
  Leer pwd
Hasta Que pwd = "secreto"
Escribir "Acceso permitido"
  
```

Diagrama paso a paso



- 1) Se entra directamente al cuerpo (no hay condición previa).
- 2) Se ejecutan las instrucciones del cuerpo: Escribir, Leer...
- 3) Se llega al **rombo** de abajo y se comprueba `pwd = "secreto"`.
- 4) Si CIERTA (Sí): se sale del bucle.
- 5) Si FALSA (No): la flecha amarilla vuelve al INICIO del cuerpo y se repite.

3.10 Para ... Hasta ... Hacer / FinPara — bucle por contador

¿Para qué sirve? Repite un bloque un número **fijo** de veces, usando una variable contador. Es el bucle más típico cuando sabes exactamente cuántas iteraciones quieres.

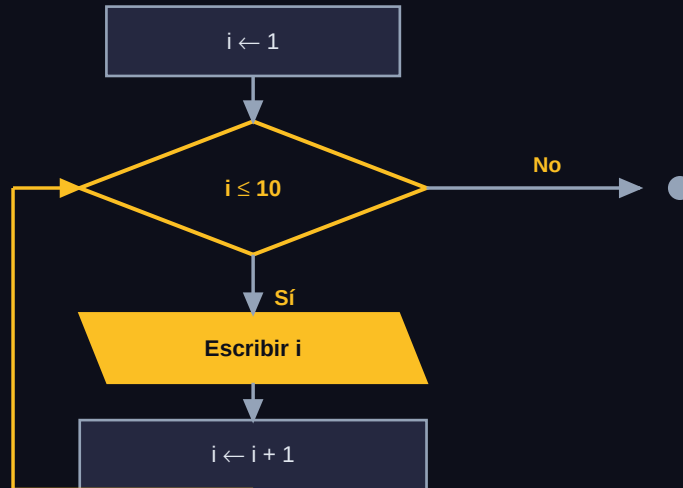
```

Definir i Como Entero
Para i <- 1 Hasta 10 Hacer
  Escribir i
FinPara

// Con paso negativo (cuenta atrás del 5 al 1):
Para i <- 5 Hasta 1 Con Paso -1 Hacer
  Escribir i
FinPara
  
```

Por defecto el paso es +1. Con `Con Paso n` puedes contar de 2 en 2, de 10 en 10, o hacia atrás con un paso negativo.

Diagrama paso a paso



- 1) **Inicialización:** la variable contador toma su valor inicial (en el ejemplo, $i \leftarrow 1$).
- 2) Se llega al rombo y se comprueba si $i \leq \text{valor_final}$. Si NO, sale por la derecha y termina.
- 3) Si SÍ: se ejecuta el cuerpo (Escribir i).
- 4) Al terminar el cuerpo, **se incrementa el contador** ($i \leftarrow i + \text{paso}$).
- 5) Vuelve al rombo y se repite.

3.11 Dimension $v[N]$ — arreglos / vectores

¿Para qué sirve? Crea una "caja con muchos compartimentos" — un **vector** (o arreglo). En lugar de tener variables sueltas para cada valor, las agrupas todas bajo un solo nombre y las accedes por número (**índice**).

■ En PSeInt, los índices van de **1 a N** (no de 0 como en otros lenguajes).

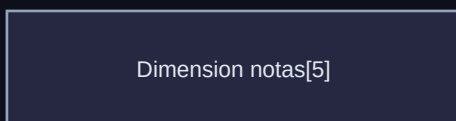
```

Definir i Como Entero
Dimension notas[5]

Para i <- 1 Hasta 5 Hacer
  Escribir "Nota del alumno ", i, ":"
  Leer notas[i]
FinPara
  
```

Ahora $\text{notas}[1]$, $\text{notas}[2]$, ..., $\text{notas}[5]$ son cinco "cajas" independientes a las que puedes acceder por índice.

En el diagrama, Dimension es un rectángulo más:



Capítulo 4 — Tu primer programa completo

Vamos a juntar todo lo aprendido en un programa pequeño pero completo: "¿Es par o impar?". El programa pide un número y dice si es par o impar.

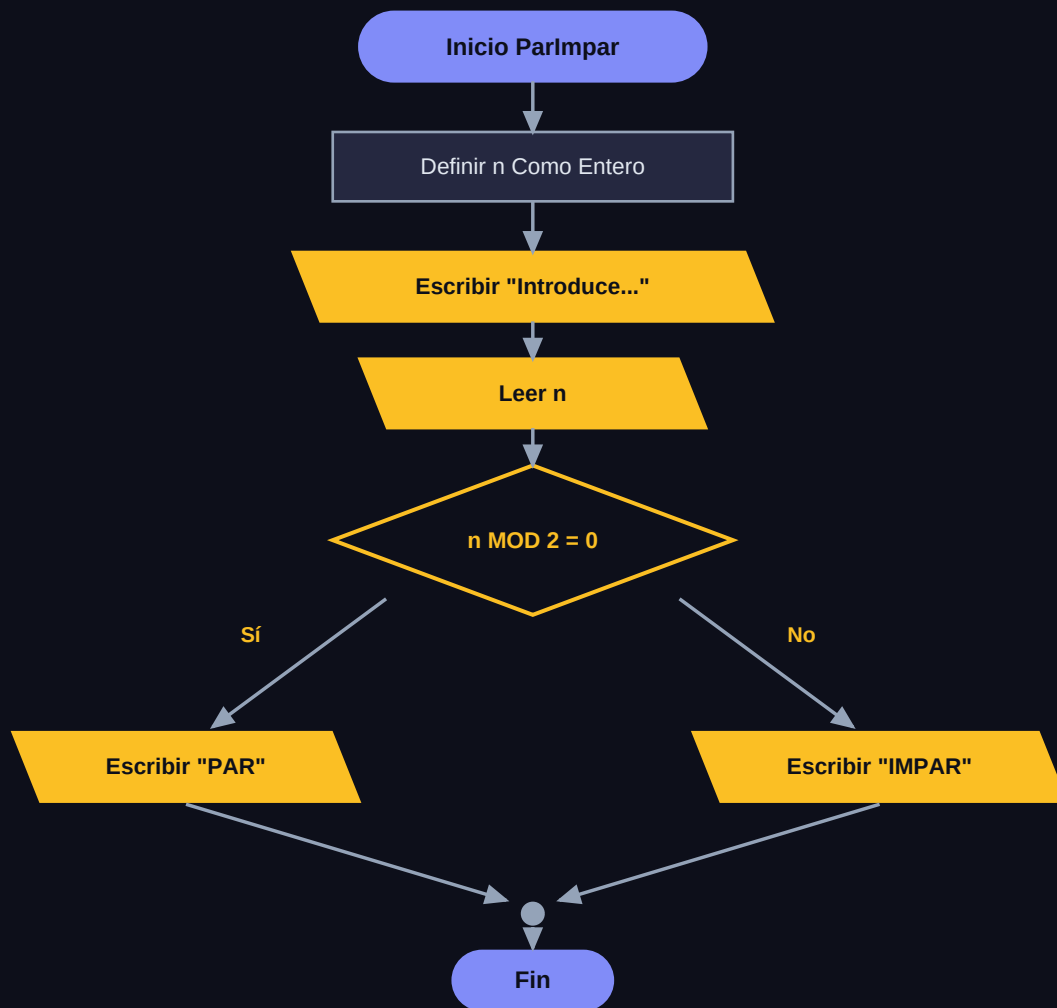
Cómo se piensa el algoritmo

1. Pedir al usuario un número entero.
2. Calcular el resto al dividir entre 2 (el operador MOD).
3. Si el resto es 0 → es par. Si no → es impar.
4. Mostrar el resultado.

El pseudocódigo

```
Proceso ParImpar
Definir n Como Entero
Escribir "Introduce un número entero:"
Leer n
Si n MOD 2 = 0 Entonces
Escribir n, " es PAR"
SiNo
Escribir n, " es IMPAR"
FinSi
FinProceso
```

El diagrama de flujo



Recorrido del diagrama paso a paso

- 1) Entramos por el óvalo **Inicio ParImpar**.
- 2) Reservamos la variable **n** de tipo Entero.
- 3) El paralelogramo **Escribir "Introduce..."** muestra el mensaje.
- 4) El paralelogramo **Leer n** espera a que el usuario introduzca un número.
- 5) El rombo evalúa $n \text{ MOD } 2 = 0$.
- 6) Si la condición es CIERTA (rama "Sí"): muestra que es PAR.
- 7) Si es FALSA (rama "No"): muestra que es IMPAR.
- 8) Las dos ramas se reúnen en el punto de unión.
- 9) Salimos por el óvalo **Fin**.

Ahora cárgalo en **PseudoWeb**: pulsa "+ Nuevo" y escríbelo, o pega esto en el editor. Pulsa  Ejecutar y prueba con varios números. Después pulsa  Diagrama y compara el diagrama que genera la app con el que acabas de ver aquí — verás que es el mismo.

PARTE I — Sobre PSeInt y PseudoWeb

1.1 ¿Qué es PSeInt? Descarga oficial

PSeInt (Pseudo-código Interpretador) es un programa de escritorio creado por Pablo Novara en la Universidad Nacional del Litoral (Argentina, ~2003). Permite escribir, ejecutar y depurar algoritmos en pseudocódigo en español. Es la herramienta de referencia en cursos universitarios de introducción a la programación en toda Latinoamérica y España.

Software libre (GPL), multiplataforma. Descarga oficial:

Sistema operativo	Enlace de descarga	Notas
Windows 7 / 10 / 11	sourceforge.net/projects/pseint/files/ · busca "Windows"	Instalador .exe (~30 MB) o ZIP portable
macOS 10.13+	sourceforge.net/projects/pseint/files/ · busca "MacOS"	Archivo .dmg para arrastrar a Aplicaciones
Linux x86_64	sourceforge.net/projects/pseint/files/ · busca "Linux"	Applimage portable o .tar.gz. En Fedora: <code>sudo dnf install pseint</code> (en algunos repos)
Página oficial	https://pseint.sourceforge.io/	Documentación, capturas, foro de ayuda

Verifica siempre la integridad del fichero comparando el SHA-256 antes de instalar; la página de SourceForge publica los hashes oficiales. Evita descargar PSeInt desde sitios que no sean SourceForge o pseint.sourceforge.io.

1.2 ¿Qué es PseudoWeb?

PseudoWeb es un IDE web complementario a PSeInt: se abre con doble clic sobre `index.html` en cualquier navegador moderno (Firefox, Chrome, Edge, Safari) y no requiere instalación. Está pensado como apoyo en clase, en aulas sin permisos de instalación, o cuando se trabaja desde un ordenador prestado.

Diferencias resumidas:

Aspecto	PSeInt (escritorio)	PseudoWeb (navegador)
Instalación	Sí (Win/macOS/Linux)	No: doble clic en HTML
Sintaxis	Pseudocódigo PSeInt	Compatible PSeInt (con limitaciones)
Diagrama de flujo	Sí, automático	No (pendiente)
Depurador paso a paso	Sí	No (pendiente)
Funciones definidas por el usuario	Sí	No (solo built-in)
Exportar a C++ y otros lenguajes	Sí	No
Sistema de ejercicios corregidos	No nativo	Sí: 36 ejercicios, nota 0-10
Autoguardado	Manual	Automático en localStorage

Recomendación: usa PSeInt en casa o en aulas con instalación permitida (es más completo); usa PseudoWeb para practicar ejercicios con corrección automática y para escenarios donde no puedas instalar nada. La sintaxis es compatible: el código que escribes en uno se ejecuta casi siempre tal cual en el otro.

1.3 Atajos y flujo de trabajo en PseudoWeb

Atajo / botón	Qué hace
Ctrl + Enter ó F5	Ejecuta el programa actual
Tab	Inserta 4 espacios (indentación)
Botón "+ Nuevo"	Borra el editor y carga la plantilla Proceso/FinProceso
Selector "Cargar ejemplo"	38 ejemplos en 12 grupos didácticos
Botón "■ Ejercicios"	Catálogo de 36 ejercicios con corrección automática
Botón "Limpiar salida"	Vacía el panel derecho sin afectar al editor
Escape (en modales)	Cierra el modal abierto

El editor guarda su contenido automáticamente cada 400 ms en el almacenamiento del navegador (localStorage). Si recargas la página o vuelves al día siguiente, tu último programa sigue ahí.

PARTE II — Referencia de sintaxis

2.1 Estructura del programa

Todo programa empieza con `Proceso NombreDelPrograma` y termina con `FinProceso`. `Algoritmo/FinAlgoritmo` es un alias aceptado. El nombre del proceso no afecta a la ejecución, pero es buena práctica que describa la tarea.

```
Proceso HolaMundo
Escribir "¡Hola, mundo!"
FinProceso
```

2.2 Variables y tipos

Las variables se declaran con `Definir nombre Como Tipo`. Se pueden declarar varias a la vez separadas por coma.

Tipo	Valores que admite	Ejemplo
Entero	Números enteros con signo. Truncan al asignar.	Definir n Como Entero
Real	Números con decimales (coma flotante)	Definir x Como Real
Numero	Alias genérico para Real	Definir y Como Numero
Caracter / Cadena / Texto	Cadenas de caracteres entre comillas dobles	Definir nombre Como Cadena
Logico	Verdadero o Falso	Definir ok Como Logico

La asignación se hace con `<-` (no `=`). Las variables declaradas como Entero truncan automáticamente al asignar: `n <- 7 / 2` deja `n = 3`.

2.3 Entrada / Salida

`Escribir` imprime uno o más valores (separados por comas). `Leer` espera a que el usuario teclee un valor.

```
Definir nombre Como Cadena
Definir edad Como Entero
Escribir "¿Cómo te llamas?"
Leer nombre
Escribir "¿Cuántos años tienes?"
Leer edad
Escribir "Hola, ", nombre, ". Tienes ", edad, " años."
```

2.4 Operadores

Categoría	Operadores	Ejemplo
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>^</code> <code>MOD</code>	<code>a MOD b</code> $\rightarrow$ resto de a entre b
Relacionales	<code>=</code> <code><></code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	Si <code>a >= 5</code> Entonces ...
Lógicos	<code>Y</code> <code>O</code> <code>NO</code>	Si <code>a > 0 Y b > 0</code> Entonces ...

Asignación	<-	x <- 5 + 3
------------	----	------------

*Precedencia (de mayor a menor): unario "–" · ^ · * · /, MOD · +, – · relacionales · NO · Y · O. Los paréntesis se evalúan primero — úsalos si dudas.*

2.5 Estructuras de control

Si / SiNo / FinSi

```
Si edad >= 18 Entonces
  Escribir "Mayor de edad"
SiNo
  Escribir "Menor de edad"
FinSi
```

Segun / FinSegun — selector múltiple

```
Segun opcion Hacer
1: Escribir "Uno"
2, 3: Escribir "Dos o tres"
De Otro Modo: Escribir "Cualquier otro"
FinSegun
```

Mientras / FinMientras — comprueba al principio

```
i <- 1
Mientras i <= 10 Hacer
  Escribir i
  i <- i + 1
FinMientras
```

Repetir / Hasta Que — comprueba al final (se ejecuta ≥ 1 vez)

```
Repetir
  Escribir "Introduce 0 para salir:"
  Leer n
Hasta Que n = 0
```

Para — iteración con contador

```
Para i <- 1 Hasta 10 Hacer
  Escribir i
FinPara

// con paso negativo:
Para i <- 10 Hasta 1 Con Paso -1 Hacer
  Escribir i
FinPara
```

2.6 Arreglos (Dimension)

Dimension v[N] reserva memoria para un vector de N elementos. Los índices van de 1 a N (no de 0 como en muchos lenguajes).

```

Dimension v[10]
Para i <- 1 Hasta 10 Hacer
Leer v[i]
FinPara
// luego usas v[1], v[2], ..., v[10]

```

2.7 Funciones built-in

Categoría	Funciones	Notas
Matemáticas	RC, Trunc, Redon, Abs, Ln, Exp, Sen, Cos, Tan	Ángulos en radianes
Aleatorias	Azar(N) $\rightarrow$ 0..N-1 Aleatorio(a, b) $\rightarrow$ a..b inclusive	Sin semilla controlable
Cadenas	Longitud, Mayusculas, Minusculas, Subcadena(s, i, j), Concatenar, ConvertirANumero, ConvertirATexto	Subcadena es 1-indexada e inclusiva
Constante	PI	Se usa sin paréntesis

PARTE III — Catálogo de 36 ejercicios

3.1 Cómo se evalúa cada ejercicio

Cada ejercicio tiene 10 casos de prueba (combinaciones de entradas con su salida esperada). El grader ejecuta tu programa una vez por cada caso, alimentándole las entradas, y compara la última línea (o las últimas N líneas) de salida con la esperada.

Aciertos	Nota	Indicador
10 de 10	10 / 10 — perfecto	■
5 a 9	5..9 / 10 — casi	■
1 a 4	1..4 / 10 — lejos	■
Sin intentar	— / 10	■

La comparación es estricta: "Suma: 10" no coincide con "suma: 10" (mayúscula) ni con "Suma:10" (sin espacio). Lee con cuidado el enunciado y respeta los espacios y mayúsculas exactos. Tu programa *Sí* puede imprimir prompts antes (p. ej. "Introduce N:") — solo se evalúa la última línea.

Los ejercicios del grupo ⑩ (Aleatorios) NO comprueban un valor exacto (sería imposible). Verifican el formato de la salida y que el resultado esté en el rango matemáticamente válido.

3.2 ① Básico

Programas con entrada/salida y una operación aritmética. Ideal para familiarizarse con Definir, Leer y Escribir.

Perímetro del cuadrado (id: perimetro_cuadrado · 10 casos)

Pide al usuario la longitud del lado de un cuadrado y calcula su perímetro ($4 \times \text{lado}$).

Como ÚLTIMA línea debes imprimir exactamente:

Perímetro: X

donde X es el valor calculado. Puedes (y debes) imprimir prompts antes — solo se evalúa la última línea.

Pistas:

- El perímetro de un cuadrado es $4 \times \text{lado}$
- Usa Definir lado, p Como Real
- Imprime así: Escribir "Perímetro: ", p

Doble y triple (id: doble_triple · 10 casos)

Pide un número y muestra su doble y su triple en una sola línea.

Como ÚLTIMA línea imprime exactamente:

Doble: X, Triple: Y

Pistas:

- El doble es $2 * n$
- El triple es $3 * n$
- Imprime los dos valores en la misma Escribir, separados por coma

Minutos a segundos (id: min_a_seg · 10 casos)

Pide un número de minutos y muestra cuántos segundos son (1 minuto = 60 segundos).

Como ÚLTIMA línea imprime exactamente:
Segundos: X

Pistas:

- Multiplica por 60
- No olvides el formato "Segundos: "

3.3 ② Condicional simple

Un único Si/SiNo. Aprende a leer una condición y decidir entre dos caminos.

¿Es múltiplo de 3? (id: multiplo_3 · 10 casos)

Pide un número entero y muestra como ÚLTIMA línea:

SI — si es múltiplo de 3

NO — si no lo es

(Recuerda: 0 sí es múltiplo de 3 porque $0 = 3 \times 0$.)

Pistas:

- Usa el operador MOD: $n \text{ MOD } 3 = 0$ indica múltiplo de 3
- Solo necesitas un Si/SiNo
- Imprime "SI" o "NO" (mayúsculas, sin nada más)

¿Aprobado? (id: aprobado · 10 casos)

Pide una nota (0..10) y muestra "APROBADO" si la nota es ≥ 5 , o "SUSPENSO" si es menor.

Como ÚLTIMA línea imprime exactamente:
APROBADO o SUSPENSO

Pistas:

- Un Si simple con la condición $\text{nota} \geq 5$
- "APROBADO" y "SUSPENSO" en mayúsculas, sin nada más

Mayor de dos números (id: mayor_dos · 10 casos)

Pide dos números. Muestra el mayor de ambos. Si son iguales, muestra "Son iguales".

Como ÚLTIMA línea imprime exactamente:

Mayor: X (si uno es mayor)

Son iguales (si $a = b$)

Pistas:

- Si $a > b$ Entonces ... SiNo Si $a < b$... SiNo Son iguales
- Cuidado con el caso de empate

3.4 ③ Condicional múltiple

Si encadenados o anidados. Cuando hay más de dos caminos posibles.

Clasificar edad (id: clasificar_edad · 10 casos)

Pide la edad de una persona y clasifícala. Como ÚLTIMA línea imprime exactamente UNA de estas palabras:

NIÑO — si $\text{edad} \leq 12$

ADOLESCENTE — si $13 \leq \text{edad} \leq 17$

ADULTO — si $18 \leq \text{edad} \leq 64$

MAYOR — si $\text{edad} \geq 65$

Asume $\text{edad} \geq 0$.

Pistas:

- Encadena varios Si...SiNo o usa Si anidados
- Cuidado con los límites: 12 es NIÑO, 13 es ADOLESCENTE
- Las cuatro palabras van en mayúsculas, sin nada más

Calificación (nota → texto) (id: calificacion · 10 casos)

Pide una nota (0..10) y muestra una categoría:

$\text{nota} < 5 \rightarrow$ INSUFICIENTE

$\text{nota} < 6 \rightarrow$ SUFICIENTE

$\text{nota} < 7 \rightarrow$ BIEN

$\text{nota} < 9 \rightarrow$ NOTABLE

$\text{nota} \leq 10 \rightarrow$ SOBRESALIENTE

Si la nota es < 0 o > 10 , imprime "FUERA DE RANGO".

Como ÚLTIMA línea imprime exactamente una de esas palabras (en mayúsculas).

Pistas:

- Encadena Si/SiNo Si en cascada
- Comprueba primero si está fuera de rango
- Cuidado con los límites: 9 es SOBRESALIENTE, 8.99 es NOTABLE

Mayor de tres números (id: mayor_tres · 10 casos)

Pide tres números y muestra el mayor de los tres.

Como ÚLTIMA línea imprime exactamente:

Mayor: X

Pistas:

- Inicializa una variable mayor con el primer valor
- Compárala con el segundo, después con el tercero
- Usa Si encadenados, no necesitas SiNo

3.5 ④ Bucle Para

Iteración cuando sabes de antemano cuántas veces vas a repetir. Ideal para sumas, tablas, factorial.

Suma de los N primeros pares (id: suma_n_pares · 10 casos)

Pide un número N y calcula la suma de los N primeros números pares positivos:
 $2 + 4 + 6 + \dots + (2 \cdot N)$

Como ÚLTIMA línea imprime exactamente:
Suma: X

Pistas:

- El i-ésimo número par positivo es $2 \cdot i$
- Usa Para $i \leq 1$ Hasta N
- Acumula en una variable suma $\leftarrow$ suma + $2 \cdot i$

Tabla de multiplicar de N (id: tabla_n · 10 casos)

Pide un número N. Imprime su tabla de multiplicar del 1 al 10 con este formato (10 líneas, una por valor):

N x 1 = X

N x 2 = X

...

N x 10 = X

No añadas nada después de la última línea.

Pistas:

- Bucle Para $i \leq 1$ Hasta 10
- Imprime: Escribir n, " x ", i, " = ", $n \cdot i$
- Sin línea final extra

Factorial (id: factorial · 10 casos)

Pide un entero $N \geq 0$ y calcula $N! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot N$. Recuerda: $0! = 1$ (por convenio).

Como ÚLTIMA línea imprime exactamente:
 $N! = X$

(El formato literal: dígito, signo de admiración, espacio, igual, espacio, valor.)

Pistas:

- Inicializa $f \leftarrow 1$
- Bucle Para $i \leq 1$ Hasta n: $f \leftarrow f \cdot i$
- Si $N = 0$, el bucle no se ejecuta y f queda en 1

3.6 ⑤ Bucle Mientras

Iteración cuando NO sabes cuántas veces, sino que dependes de una condición. Patrón centinela.

Sumar hasta introducir 0 (id: sumar_hasta_cero · 10 casos)

Pide números al usuario, uno a uno. Cuando introduzca 0, deja de pedir más y muestra la suma de todos los anteriores (el 0 final NO se suma — es el centinela).

Como ÚLTIMA línea imprime exactamente:
Suma: X

Si el primer número es 0, la suma vale 0.

Pistas:

- Inicializa `suma <- 0` y lee el primer número
- Mientras `n <> 0` Hacer: `suma <- suma + n`, leer el siguiente
- Recuerda imprimir "Suma: " antes del total

Cuenta las cifras de un número (id: cifras_numero · 10 casos)

Pide un entero $N > 0$ y cuenta cuántas cifras tiene.

Como ÚLTIMA línea imprime exactamente:
Cifras: X

Asume $N > 0$ (no se prueba con 0 ni con negativos).

Pistas:

- Inicia `cifras <- 0`
- Mientras `n > 0`: `cifras <- cifras + 1`, `n <- n / 10` (truncado por Entero)

Promedio con centinela -1 (id: promedio_centinela · 10 casos)

Pide números hasta que el usuario introduzca -1 (centinela). Calcula el promedio de los anteriores (el -1 NO se cuenta).

Como ÚLTIMA línea imprime exactamente:
Promedio: X (si hubo al menos un dato)
Sin datos (si el primer número introducido fue -1)

Pistas:

- Cuenta cuántos datos lees
- Acumula `suma <- suma + n`
- `Promedio = suma / contador`
- Antes de imprimir comprueba si `contador > 0`

3.7 ⑥ Algoritmos clásicos

Combinación de bucles y condicionales que resuelven problemas con identidad propia.

Suma de dígitos (id: suma_digitos · 10 casos)

Pide un número entero positivo y calcula la suma de todos sus dígitos.

Por ejemplo:

123 → 1+2+3 = 6

1000 → 1+0+0+0 = 1

999 → 9+9+9 = 27

Como ÚLTIMA línea imprime exactamente:

Suma digitos: X

Pistas:

- Extrae el último dígito con: `digito <- n MOD 10`
- Acumula en suma <- suma + digito
- Reduce n con: `n <- n / 10` (si n es Entero, se trunca automáticamente)
- Repite mientras `n > 0`

¿Es primo? (id: es_primo · 10 casos)

Pide un número entero $N \geq 2$ y determina si es primo (solo divisible entre 1 y sí mismo).

Como ÚLTIMA línea imprime exactamente:

PRIMO o NO PRIMO

Pistas:

- `Bandera es_primo <- Verdadero`
- Recorre divisores con Para `i <- 2` Hasta `n-1` (o hasta `RC(n)`)
- Si encuentras un divisor: `es_primo <- Falso`
- Imprime "PRIMO" o "NO PRIMO" según la bandera

Invertir un número (id: invertir_numero · 10 casos)

Pide un entero positivo y muestra el número con sus dígitos invertidos.

Ejemplo: 1234 → 4321. Los ceros a la izquierda del resultado se omiten (10 → 1, no "01").

Como ÚLTIMA línea imprime exactamente:

Invertido: X

Pistas:

- `inv <- 0`
- Mientras `n > 0`: `d <- n MOD 10`, `inv <- inv * 10 + d`, `n <- n / 10`
- Como inv es Entero, los ceros iniciales desaparecen solos

3.8 ⑦ Listas (Dimension)

Trabajar con varios valores en un solo vector. Recorrer, buscar, contar.

Contar pares e impares (id: pares_impares · 10 casos)

Pide N. A continuación pide N números enteros y guárdalos en un array. Cuenta cuántos son pares y cuántos impares.

Recuerda: 0 es par. Los negativos también pueden ser pares o impares (−2 es par, −1 es impar).

Como ÚLTIMA línea imprime exactamente:

Pares: X, Impares: Y

Máximo de un vector (id: max_vector · 10 casos)

Pide N. Después pide N números y guárdalos en un array. Muestra el valor máximo.

Como ÚLTIMA línea imprime exactamente:

Maximo: X (sin tilde para que el grader sea estricto)

Buscar un valor en un vector (id: buscar_valor · 10 casos)

Pide N. Después pide N números. Después pide un valor X a buscar. Devuelve la POSICIÓN (1-indexada) donde aparece la primera ocurrencia de X, o -1 si no aparece.

Como ÚLTIMA línea imprime exactamente:

Posicion: X (sin tilde)

Pistas:

- Usa una bandera o variable pos <- -1
- Recorre con Mientras o Para; para Mientras puedes salir cuando lo encuentres
- Asegúrate de no sobrescribir pos si ya lo encontraste

3.9 ⑧ Menús con Segun

La estructura Segun como selector múltiple, ideal para menús con varias opciones.

Día de la semana (id: día_semana · 10 casos)

Pide un número del 1 al 7 e imprime el día correspondiente. Si está fuera de rango imprime "Día no válido".

1 → Lunes 5 → Viernes
2 → Martes 6 → Sábado
3 → Miércoles 7 → Domingo
4 → Jueves

Como ÚLTIMA línea imprime el nombre del día (con sus tildes correctas) o exactamente "Día no válido".

REQUISITO: usa la estructura Segun (no Si encadenados).

Pistas:

- Usa Segun n Hacer ... FinSegun
- Para el caso fuera de rango usa "De Otro Modo:"
- Cuidado con las tildes: Miércoles, Sábado, Día

Días del mes (id: días_mes · 10 casos)

Pide el número del mes (1..12) y muestra cuántos días tiene. Asume año NO bisiesto (febrero = 28).

Como ÚLTIMA línea imprime exactamente:

Días: X (sin tilde, si mes en rango)

Mes no válido (si fuera de rango)

REQUISITO: usa Segun.

Pistas:

- Meses con 31 días: 1, 3, 5, 7, 8, 10, 12
- Meses con 30 días: 4, 6, 9, 11
- Febrero (2) = 28
- Agrupa con: 1, 3, 5, 7, 8, 10, 12: Escribir "Días: 31"

Nombre del mes (id: nombre_mes · 10 casos)

Pide un número del 1 al 12 y muestra el nombre del mes en español.

Como ÚLTIMA línea imprime exactamente uno de:
Enero, Febrero, Marzo, Abril, Mayo, Junio,
Julio, Agosto, Septiembre, Octubre, Noviembre, Diciembre,
Mes no válido

REQUISITO: usa Segun.

Pistas:

- Cada caso del Segun imprime un nombre
- Cuidado con la ortografía: Septiembre (no Setiembre)

3.10 ⑨ Repetir / Hasta Que

Bucle do-while: se ejecuta al menos una vez y comprueba la condición al final. Ideal para validar entradas.

Adivinar el 7 (id: adivinar_7 · 10 casos)

El número secreto es 7 (fijo, no aleatorio). Pide intentos al usuario hasta que adivine. Cuenta el número de intentos.

REQUISITO: usa Repetir / Hasta Que.

Como ÚLTIMA línea imprime exactamente:
Intentos: X

donde X incluye el intento acertado (si acierta a la primera, X=1).

Pistas:

- Inicializa intentos <- 0 antes del bucle
- Dentro de Repetir: Leer intento, intentos <- intentos + 1
- La condición de salida es: Hasta Que intento = 7

Validar nota 0-10 con Repetir (id: validar_nota_repetir · 10 casos)

Pide una nota hasta que esté entre 0 y 10 (ambos inclusive). Cuando la nota sea válida, imprime como ÚLTIMA línea:
Nota válida: X

REQUISITO: usa Repetir / Hasta Que (no Mientras).

Pistas:

- Repetir { Leer nota } Hasta Que nota >= 0 Y nota <= 10
- Los valores incorrectos NO se imprimen, solo se descartan

Contar hasta múltiplo de 7 (id: contar_hasta_mult7 · 10 casos)

Pide números al usuario uno a uno usando Repetir/Hasta Que. Para cuando introduzca un múltiplo de 7 (incluido el 0, que es múltiplo de 7). Cuenta CUÁNTOS números introdujo en total (incluyendo el múltiplo de 7 final).

Como ÚLTIMA línea imprime exactamente:

Contados: X

Pistas:

- `contados <- 0`
- Dentro de Repetir: `Leer n, contados <- contados + 1`
- Hasta Que `n MOD 7 = 0`

3.11 ⑩ Números aleatorios

Uso de Aleatorio() para simulaciones (dados, monedas, juegos).

Tirar N dados y sumar (id: tirar_dados · 10 casos)

Pide un número N y tira N dados (cada uno con Aleatorio(1, 6)). Suma todos los resultados.

Como ÚLTIMA línea imprime exactamente:

Suma: X

Como X depende del azar, el grader sólo verifica que el formato es correcto y que la suma está en el rango válido (entre N y 6·N, ambos inclusive).

Pistas:

- Usa la función Aleatorio(1, 6) — devuelve un entero de 1 a 6
- Acumula en `suma <- suma + Aleatorio(1, 6)`
- No olvides inicializar `suma <- 0`

■ Como usa Aleatorio, el grader comprueba que (1) el formato sea "Suma: X" y (2) que X esté en el rango [N, 6·N]. No se comprueba el valor exacto.

Lanzar N monedas (id: lanzar_monedas · 10 casos)

Pide un número N. Lanza N monedas con Aleatorio(0, 1): considera 0 = cara, 1 = cruz. Cuenta cuántas caras y cuántas cruces salen.

Como ÚLTIMA línea imprime exactamente:

Caras: X, Cruces: Y

Pistas:

- Usa Aleatorio(0, 1) dentro de un bucle Para
- Si el resultado es 0 sumas caras, si es 1 sumas cruces
- Caras + Cruces siempre tiene que dar N

■ Usa Aleatorio. El grader comprueba que el formato es "Caras: X, Cruces: Y" y que $X + Y = N$. No se comprueban los valores exactos.

Aleatorios > 50 (id: mayores_50 · 10 casos)

Pide N. Genera N números aleatorios entre 1 y 100 con Aleatorio(1, 100). Cuenta cuántos son MAYORES que 50 (estrictamente).

Como ÚLTIMA línea imprime exactamente:
Mayores: X

Pistas:

- Bucle Para i <- 1 Hasta n
- Genera r <- Aleatorio(1, 100)
- Si r > 50 entonces incrementa mayores

■ Usa Aleatorio. El grader comprueba el formato "Mayores: X" y que X está en el rango [0, N].

3.12 ■ Cadenas

Contar consonantes (id: contar_consonantes · 10 casos)

Pide una palabra escrita en minúsculas, sin acentos y sin espacios. Cuenta cuántas consonantes tiene.

Vocales = {a, e, i, o, u}. Cualquier otra letra cuenta como consonante.

Como ÚLTIMA línea imprime exactamente:
Consonantes: X

Pistas:

- Usa n <- Longitud(palabra) para saber cuántos caracteres iterar
- Saca cada letra con: c <- Subcadena(palabra, i, i)
- Comprueba si NO es vocal: c <> "a" Y c <> "e" Y ...

Invertir una palabra (id: invertir_cadena · 10 casos)

Pide una palabra y muéstrala con sus caracteres en orden inverso.

Como ÚLTIMA línea imprime exactamente:
Invertida: X

Pistas:

- Inicializa una cadena vacía: inv <- ""
- Recorre desde i = Longitud(palabra) hasta 1 con paso -1
- En cada iteración: inv <- Concatenar(inv, Subcadena(palabra, i, i))

¿Empieza por vocal? (id: empieza_vocal · 10 casos)

Pide una palabra (escrita en minúsculas, sin acentos). Determina si su primera letra es una vocal (a, e, i, o, u).

Como ÚLTIMA línea imprime exactamente:
SI o NO

Pistas:

- Obtén la primera letra: c <- Subcadena(palabra, 1, 1)
- Comprueba si c = "a" O c = "e" O ...

3.13 ■ Funciones matemáticas

Uso de la constante PI y de funciones RC, Redon, Trunc para fórmulas geométricas.

Área del círculo (id: area_circulo · 10 casos)

Pide el radio R de un círculo y calcula su área:

$$\text{área} = \pi \cdot r^2$$

Muestra el área redondeada al entero más cercano (usa la función Redon).

Como ÚLTIMA línea imprime exactamente:

Area: X

(Usa "Area" sin tilde para que el grader sea estricto.)

Pistas:

- La constante PI ya está definida (no necesitas paréntesis)
- Usa el operador ^ para el cuadrado: r^2
- Redondea con: `a <- Redon(PI * r ^ 2)`

Hipotenusa (Pitágoras) (id: hipotenusa · 10 casos)

Pide los dos catetos A y B de un triángulo rectángulo y calcula la hipotenusa: $h = \sqrt{a^2 + b^2}$.

Muestra el resultado redondeado al entero más cercano (usa Redon y RC).

Como ÚLTIMA línea imprime exactamente:

Hipotenusa: X

Pistas:

- Usa RC para la raíz cuadrada y ^ para el cuadrado
- `h <- Redon(RC(a ^ 2 + b ^ 2))`
- Aprovecha las ternas pitagóricas (3-4-5, 5-12-13, ...) para entender el caso

Distancia entre dos puntos (id: distancia_puntos · 10 casos)

Pide las coordenadas de dos puntos: (x1, y1) y (x2, y2). Calcula la distancia euclidiana:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

Muestra el resultado redondeado al entero más cercano.

Como ÚLTIMA línea imprime exactamente:

Distancia: X

Pistas:

- Lee los cuatro valores en orden: x1, y1, x2, y2
- `d <- Redon(RC((x2 - x1) ^ 2 + (y2 - y1) ^ 2))`

PARTE IV — Apéndices

4.1 Errores comunes y cómo solucionarlos

Mensaje del intérprete	Qué significa	Solución típica
Variable 'x' usada sin definir	Usaste x antes de declararla con Definir, o escribiste mal el nombre	Añade Definir x Como ... al principio del Proceso, o revisa la ortografía
Se esperaba ASIGN pero se encontró ...	Pusiste = en una asignación en lugar de <-	Cambia x = 5 por x <- 5
Se esperaba FINSI pero se encontró EOF	Te olvidaste de cerrar un Si con FinSi	Cuenta los Si/Mientras/Para abiertos y asegúrate de que cada uno tiene su FinSi/FinMientras/FinPara
Índice X fuera de rango	Accediste a v[i] con i fuera de 1..N donde N es el tamaño del Dimension	Revisa los límites de tus bucles (Para i <- 1 Hasta N, no Hasta N+1)
La condición del Si debe ser lógica	En el Si pusiste un número o cadena en lugar de una comparación	Asegúrate de usar operadores relacionales: Si x > 0 Entonces (no Si x Entonces)
Bucle ... detenido por seguridad tras 1 000 000 iteraciones	Tu Mientras o Repetir nunca termina (la condición no cambia)	Comprueba que actualizas la variable de control dentro del bucle
División por cero	Algún cálculo divide entre 0	Antes de dividir, comprueba con Si que el divisor no es 0
Valor no numérico para 'x': "..."	Declaraste x Como Entero/Real pero el usuario tecleó letras	Asume que el usuario teclea números válidos, o cambia el tipo a Cadena

4.2 Tabla resumen de constructos

Constructo	Sintaxis
Programa	Proceso Nombre ... FinProceso
Definir	Definir x, y Como Entero
Asignar	x <- expresión
Imprimir	Escribir a, b, c
Leer	Leer x Leer v[i]
Si	Si cond Entonces ... SiNo ... FinSi
Segun	Segun expr Hacer N: ... De Otro Modo: ... FinSegun
Mientras	Mientras cond Hacer ... FinMientras
Repetir	Repetir ... Hasta Que cond
Para	Para i <- a Hasta b [Con Paso s] Hacer ... FinPara
Dimension	Dimension v[N] // índices 1..N
Comentario	// el resto de la línea es comentario
Operadores	+ - * / ^ MOD = <> < > <= >= Y O NO

Booleanos	Verdadero Falso
Built-in math	RC Trunc Redon Abs Ln Exp Sen Cos Tan PI
Built-in azar	Azar(N) Aleatorio(a, b)
Built-in texto	Longitud Mayusculas Minusculas Subcadena Concatenar

PARTE V — Soluciones paso a paso de los 36 ejercicios

Esta parte está pensada como **libro de respuestas**. Si te quedas atascado en un ejercicio, busca aquí su solución de referencia. Pero **úsala como último recurso**: el aprendizaje real ocurre cuando peleas con el problema y descubres tú la solución, no cuando la copias.

Cada solución incluye: el código pseudocódigo completo (probado: pasa los 10 casos del grader), una "Idea" con el enfoque algorítmico en una o dos frases y, cuando aplica, un "Truco" con una observación útil que es fácil pasar por alto.

① Básico

Perímetro del cuadrado (id: perimetro_cuadrado · pasa 10/10 casos)

```
Proceso Perimetro
Definir lado, p Como Real
Escribir "Lado:"
Leer lado
p <- 4 * lado
Escribir "Perímetro: ", p
FinProceso
```

Idea: El perímetro de un cuadrado es $4 \times \text{lado}$. Una sola operación aritmética entre Leer y Escribir.

Doble y triple (id: doble_triple · pasa 10/10 casos)

```
Proceso DobleTriple
Definir n Como Real
Leer n
Escribir "Doble: ", 2 * n, ", Triple: ", 3 * n
FinProceso
```

Idea: No hace falta variables intermedias: Escribir admite expresiones directamente.

Truco: Imprimir varios valores con comas los concatena en la misma línea.

Minutos a segundos (id: min_a_seg · pasa 10/10 casos)

```
Proceso MinASeg
Definir min, seg Como Real
Leer min
seg <- min * 60
Escribir "Segundos: ", seg
FinProceso
```

Idea: Un minuto son 60 segundos. Multiplica.

② Condicional simple

¿Es múltiplo de 3? (id: multiplo 3 · pasa 10/10 casos)

```
Proceso Multiplo3
Definir n Como Entero
Leer n
Si n MOD 3 = 0 Entonces
Escribir "SI"
SiNo
Escribir "NO"
FinSi
FinProceso
```

Idea: El operador MOD da el resto de la división. Si el resto entre 3 es 0, n es múltiplo de 3.

Truco: Recuerda: $0 \text{ MOD } 3 = 0$, así que el 0 sí es múltiplo de 3.

¿Aprobado? (id: aprobado · pasa 10/10 casos)

```
Proceso Aprobado
Definir nota Como Real
Leer nota
Si nota >= 5 Entonces
Escribir "APROBADO"
SiNo
Escribir "SUSPENSO"
FinSi
FinProceso
```

Idea: Una sola comparación ≥ 5 . El Si/SiNo elige el mensaje.

Mayor de dos números (id: mayor dos · pasa 10/10 casos)

```
Proceso MayorDos
Definir a, b Como Real
Leer a
Leer b
Si a > b Entonces
Escribir "Mayor: ", a
SiNo
Si b > a Entonces
Escribir "Mayor: ", b
SiNo
Escribir "Son iguales"
FinSi
FinSi
FinProceso
```

Idea: Tres casos exclusivos: $a > b$, $b > a$, o iguales. Anida un Si dentro del SiNo del primero.

Truco: El caso de empate hay que tratarlo aparte — no basta con "si no es mayor, es menor".

③ Condicional múltiple

Clasificar edad (id: clasificar edad · pasa 10/10 casos)

```
Proceso ClasificarEdad
Definir edad Como Entero
Leer edad
Si edad <= 12 Entonces
  Escribir "NIÑO"
SiNo
  Si edad <= 17 Entonces
    Escribir "ADOLESCENTE"
  SiNo
    Si edad <= 64 Entonces
      Escribir "ADULTO"
    SiNo
      Escribir "MAYOR"
  FinSi
FinSi
FinSi
FinSi
FinProceso
```

Idea: Cascada de Si encadenados en orden ascendente. Cada SiNo cubre los casos restantes.

Truco: Si compruebas edad <= 12 primero, no necesitas decir "edad >= 0" en los siguientes — ya lo sabes.

Calificación (nota → texto) (id: calificacion · pasa 10/10 casos)

```
Proceso Calificacion
Definir nota Como Real
Leer nota
Si nota < 0 O nota > 10 Entonces
  Escribir "FUERA DE RANGO"
SiNo
  Si nota < 5 Entonces
    Escribir "INSUFICIENTE"
  SiNo
    Si nota < 6 Entonces
      Escribir "SUFICIENTE"
    SiNo
      Si nota < 7 Entonces
        Escribir "BIEN"
      SiNo
        Si nota < 9 Entonces
          Escribir "NOTABLE"
        SiNo
          Escribir "SOBRESALIENTE"
    FinSi
  FinSi
FinSi
FinSi
FinSi
FinSi
FinProceso
```

Idea: Comprueba primero el caso fuera de rango (con O lógico). Luego cascada de rangos de menor a mayor.

Truco: Cada Si comprueba sólo el límite superior — el inferior ya lo garantiza el SiNo del anterior.

Mayor de tres números (id: mayor tres · pasa 10/10 casos)

```
Proceso MayorTres
Definir a, b, c, mayor Como Real
Leer a
Leer b
Leer c
mayor <- a
Si b > mayor Entonces
mayor <- b
FinSi
Si c > mayor Entonces
mayor <- c
FinSi
Escribir "Mayor: ", mayor
FinProceso
```

Idea: Patrón clásico: asume que el primero es el mayor; compara con los siguientes y actualiza si encuentra uno más grande.

④ Bucle Para

Suma de los N primeros pares (id: suma n pares · pasa 10/10 casos)

```
Proceso SumaPares
Definir n, i, suma Como Entero
Leer n
suma <- 0
Para i <- 1 Hasta n Hacer
suma <- suma + 2 * i
FinPara
Escribir "Suma: ", suma
FinProceso
```

Idea: El i-ésimo par positivo es $2 \cdot i$ (el 1º es 2, el 2º es 4, ...). Bucle Para acumula.

Truco: Inicializa el acumulador a 0 ANTES del Para, no dentro.

Tabla de multiplicar de N (id: tabla n · pasa 10/10 casos)

```
Proceso TablaN
Definir n, i Como Entero
Leer n
Para i <- 1 Hasta 10 Hacer
Escribir n, " x ", i, " = ", n * i
FinPara
FinProceso
```

Idea: Para i de 1 a 10. Imprime n por i. Sin acumulador.

Factorial (id: factorial · pasa 10/10 casos)

```
Proceso Factorial
Definir n, i, f Como Entero
Leer n
f <- 1
Para i <- 1 Hasta n Hacer
  f <- f * i
FinPara
Escribir n, "! = ", f
FinProceso
```

Idea: f comienza en 1. Multiplica por 1, 2, ..., n. Si n=0, el Para no se ejecuta y f queda en 1 (que es 0! por convenio).

Truco: Inicializar el acumulador a 1 (neutro de la multiplicación), NO a 0.

⑤ Bucle Mientras

Sumar hasta introducir 0 (id: sumar_hasta_cero · pasa 10/10 casos)

```
Proceso SumarHasta0
Definir n, suma Como Real
suma <- 0
Leer n
Mientras n <> 0 Hacer
  suma <- suma + n
  Leer n
FinMientras
Escribir "Suma: ", suma
FinProceso
```

Idea: Patrón centinela: lee un número ANTES del Mientras, luego lee el siguiente DENTRO del bucle.

Truco: Si pusieras Leer DESPUÉS de la suma, sumarías el 0 final.

Cuenta las cifras de un número (id: cifras_numero · pasa 10/10 casos)

```
Proceso Cifras
Definir n, cifras Como Entero
Leer n
cifras <- 0
Mientras n > 0 Hacer
  cifras <- cifras + 1
  n <- n / 10
FinMientras
Escribir "Cifras: ", cifras
FinProceso
```

Idea: Cada división entre 10 quita una cifra (truncando, gracias al tipo Entero). Cuenta cuántas veces puedes dividir.

Truco: En PseudoWeb, cuando n es Entero, "n <- n / 10" trunca automáticamente.

Promedio con centinela -1 (id: promedio centinela · pasa 10/10 casos)

```
Proceso Promedio
Definir n, suma, prom Como Real
Definir cont Como Entero
suma <- 0
cont <- 0
Leer n
Mientras n <> -1 Hacer
    suma <- suma + n
    cont <- cont + 1
Leer n
FinMientras
Si cont > 0 Entonces
    prom <- suma / cont
    Escribir "Promedio: ", prom
SiNo
    Escribir "Sin datos"
FinSi
FinProceso
```

Idea: Mismo patrón centinela que el anterior pero con dos acumuladores: suma y contador. División sólo si contador > 0.

Truco: NUNCA dividas entre el contador sin comprobar que es > 0: la división por cero es un error.

⑥ Algoritmos clásicos

Suma de dígitos (id: suma digitos · pasa 10/10 casos)

```
Proceso SumaDigitos
Definir n, suma, digito Como Entero
Leer n
suma <- 0
Mientras n > 0 Hacer
    digito <- n MOD 10
    suma <- suma + digito
    n <- n / 10
FinMientras
Escribir "Suma digitos: ", suma
FinProceso
```

Idea: $n \bmod 10$ da el último dígito. $n / 10$ (truncado) quita el último dígito. Repite hasta que n llegue a 0.

Truco: Si n vale 0, el Mientras no se ejecuta y suma queda en 0 — justo lo que queremos.

¿Es primo? (id: es primo · pasa 10/10 casos)

```
Proceso EsPrimo
Definir n, i Como Entero
Definir esp Como Logico
Leer n
esp <- Verdadero
Para i <- 2 Hasta n - 1 Hacer
Si n MOD i = 0 Entonces
esp <- Falso
FinSi
FinPara
Si esp Entonces
Escribir "PRIMO"
SiNo
Escribir "NO PRIMO"
FinSi
FinProceso
```

Idea: Variable bandera (esp). Prueba todos los divisores del 2 al n-1. Si alguno cabe exacto, no es primo.

Truco: Una optimización (no necesaria aquí): basta probar hasta $\sqrt{n}$. Pero con n pequeños no importa.

Invertir un número (id: invertir numero · pasa 10/10 casos)

```
Proceso InvertirNum
Definir n, inv, digito Como Entero
Leer n
inv <- 0
Mientras n > 0 Hacer
digito <- n MOD 10
inv <- inv * 10 + digito
n <- n / 10
FinMientras
Escribir "Invertido: ", inv
FinProceso
```

Idea: Saca el último dígito y "empújalo" a la izquierda del invertido (multiplica por 10).

Truco: Los ceros a la izquierda del invertido desaparecen solos porque inv es Entero ($10 \rightarrow 1$, no "01").

⑦ Listas (Dimension)

Contar pares e impares (id: pares impares · pasa 10/10 casos)

```
Proceso ParesImpares
Definir n, i, pares, impares Como Entero
Dimension v[100]
Leer n
Para i <- 1 Hasta n Hacer
Leer v[i]
FinPara
pares <- 0
impares <- 0
Para i <- 1 Hasta n Hacer
Si v[i] MOD 2 = 0 Entonces
pares <- pares + 1
SiNo
impares <- impares + 1
FinSi
FinPara
Escribir "Pares: ", pares, ", Impares: ", impares
FinProceso
```

Idea: Dos pasadas con Para: una para leer, otra para contar. $v[i] \text{ MOD } 2$ distingue par/impar.

Truco: En PseudoWeb, el MOD funciona también con negativos: $-2 \text{ MOD } 2 = 0$ (par), $-1 \text{ MOD } 2 = 1$ (impar).

Máximo de un vector (id: max vector · pasa 10/10 casos)

```
Proceso MaxVector
Definir n, i, max Como Entero
Dimension v[100]
Leer n
Para i <- 1 Hasta n Hacer
Leer v[i]
FinPara
max <- v[1]
Para i <- 2 Hasta n Hacer
Si v[i] > max Entonces
max <- v[i]
FinSi
FinPara
Escribir "Maximo: ", max
FinProceso
```

Idea: Inicializa max con el primer elemento. Compara con los demás (de 2 a n) y actualiza si encuentras uno mayor.

Truco: NO inicialices max a 0: si todos los valores son negativos, fallaría.

Buscar un valor en un vector (id: buscar valor · pasa 10/10 casos)

```
Proceso Buscar
Definir n, i, buscar, pos Como Entero
Dimension v[100]
Leer n
Para i <- 1 Hasta n Hacer
Leer v[i]
FinPara
Leer buscar
pos <- -1
i <- 1
Mientras i <= n Y pos = -1 Hacer
Si v[i] = buscar Entonces
pos <- i
FinSi
i <- i + 1
FinMientras
Escribir "Posicion: ", pos
FinProceso
```

Idea: pos comienza en -1 (no encontrado). Mientras quedan elementos Y aún no se encontró, busca. Al encontrarlo guarda i en pos y la condición pos=-1 deja de cumplirse.

Truco: Usar Mientras (no Para) para poder cortar el bucle cuando se encuentra la primera ocurrencia.

⑧ Menús con Segun

Día de la semana (id: día semana · pasa 10/10 casos)

```
Proceso DiaSemana
Definir n Como Entero
Leer n
Segun n Hacer
1: Escribir "Lunes"
2: Escribir "Martes"
3: Escribir "Miércoles"
4: Escribir "Jueves"
5: Escribir "Viernes"
6: Escribir "Sábado"
7: Escribir "Domingo"
De Otro Modo: Escribir "Día no válido"
FinSegun
FinProceso
```

Idea: Un caso del Segun por cada día. "De Otro Modo" captura cualquier número fuera de 1..7.

Truco: Atento a las tildes: Miércoles, Sábado, "Día no válido" — la corrección es estricta.

Días del mes (id: dias mes · pasa 10/10 casos)

```
Proceso DiasMes
Definir mes Como Entero
Leer mes
Segun mes Hacer
1, 3, 5, 7, 8, 10, 12: Escribir "Dias: 31"
4, 6, 9, 11: Escribir "Dias: 30"
2: Escribir "Dias: 28"
De Otro Modo: Escribir "Mes no válido"
FinSegun
FinProceso
```

Idea: Agrupa varios valores en un mismo caso separándolos por coma — más limpio que repetir Escribir.

Truco: Los meses de 31 días: enero, marzo, mayo, julio, agosto, octubre, diciembre.

Nombre del mes (id: nombre mes · pasa 10/10 casos)

```
Proceso NombreMes
Definir mes Como Entero
Leer mes
Segun mes Hacer
1: Escribir "Enero"
2: Escribir "Febrero"
3: Escribir "Marzo"
4: Escribir "Abril"
5: Escribir "Mayo"
6: Escribir "Junio"
7: Escribir "Julio"
8: Escribir "Agosto"
9: Escribir "Septiembre"
10: Escribir "Octubre"
11: Escribir "Noviembre"
12: Escribir "Diciembre"
De Otro Modo: Escribir "Mes no válido"
FinSegun
FinProceso
```

Idea: Un caso por mes. Estilo "tabla de búsqueda" donde el Segun hace de diccionario.

Truco: Septiembre (no "Setiembre"). El grader es estricto con la ortografía.

⑨ Repetir / Hasta Que

Adivinar el 7 (id: adivinar 7 · pasa 10/10 casos)

```
Proceso Adivinar7
Definir intento, intentos Como Entero
intentos <- 0
Repetir
Leer intento
intentos <- intentos + 1
Hasta Que intento = 7
Escribir "Intentos: ", intentos
FinProceso
```

Idea: Repetir/Hasta Que ejecuta al menos una vez. Cuenta cada lectura. Sale cuando intento=7.

Truco: Si pones contadores ANTES de Leer, contarías mal. Aquí: primero leer, luego contar — y el contador incluye el acierto final.

Validar nota 0-10 con Repetir (id: validar nota repetir · pasa 10/10 casos)

```
Proceso ValidarNota
Definir nota Como Real
Repetir
Leer nota
Hasta Que nota >= 0 Y nota <= 10
Escribir "Nota válida: ", nota
FinProceso
```

Idea: Repetir asegura que se pide al menos una vez. La condición de salida usa Y para que la nota esté en el rango [0, 10].

Truco: Más limpio que Mientras: no necesitas inicializar nota con un valor "imposible".

Contar hasta múltiplo de 7 (id: contar hasta mult7 · pasa 10/10 casos)

```
Proceso ContarHasta7
Definir n, contados Como Entero
contados <- 0
Repetir
Leer n
contados <- contados + 1
Hasta Que n MOD 7 = 0
Escribir "Contados: ", contados
FinProceso
```

Idea: Cuenta cada número leído, incluido el múltiplo de 7 final. La condición de salida es $n \text{ MOD } 7 = 0$.

Truco: Cuidado: 0 es múltiplo de 7 ($0 = 7 \cdot 0$). Si el usuario introduce 0 a la primera, el bucle termina con $\text{contados} = 1$.

⑩ Números aleatorios

Tirar N dados y sumar (id: tirar dados · pasa 10/10 casos)

```
Proceso TirarDados
Definir n, i, suma Como Entero
Leer n
suma <- 0
Para i <- 1 Hasta n Hacer
suma <- suma + Aleatorio(1, 6)
FinPara
Escribir "Suma: ", suma
FinProceso
```

Idea: $\text{Aleatorio}(1, 6)$ simula un dado. Para acumula N tiradas.

Truco: La suma siempre estará entre N (todo 1s) y $6N$ (todo 6s) — el grader chequea ese rango.

Lanzar N monedas (id: lanzar_monedas · pasa 10/10 casos)

```
Proceso Monedas
Definir n, i, r, caras, cruces Como Entero
Leer n
caras <- 0
cruces <- 0
Para i <- 1 Hasta n Hacer
  r <- Aleatorio(0, 1)
  Si r = 0 Entonces
    caras <- caras + 1
  SiNo
    cruces <- cruces + 1
FinSi
FinPara
Escribir "Caras: ", caras, ", Cruces: ", cruces
FinProceso
```

Idea: Aleatorio(0, 1) da 0 (cara) o 1 (cruz). Dos contadores. La suma caras+cruces siempre es N.

Aleatorios > 50 (id: mayores_50 · pasa 10/10 casos)

```
Proceso Mayores50
Definir n, i, r, mayores Como Entero
Leer n
mayores <- 0
Para i <- 1 Hasta n Hacer
  r <- Aleatorio(1, 100)
  Si r > 50 Entonces
    mayores <- mayores + 1
  FinSi
FinPara
Escribir "Mayores: ", mayores
FinProceso
```

Idea: Genera N aleatorios entre 1 y 100. Cuenta solo los que pasan de 50.

Truco: Estadísticamente, mayores $\approx$ N/2, pero el grader sólo comprueba que esté en [0, N].

Cadenas

Contar consonantes (id: contar_consonantes · pasa 10/10 casos)

```
Proceso Consonantes
Definir palabra, c Como Cadena
Definir i, n, cont Como Entero
Leer palabra
n <- Longitud(palabra)
cont <- 0
Para i <- 1 Hasta n Hacer
  c <- Subcadena(palabra, i, i)
  Si c <> "a" Y c <> "e" Y c <> "i" Y c <> "o" Y c <> "u" Entonces
    cont <- cont + 1
  FinSi
FinPara
Escribir "Consonantes: ", cont
FinProceso
```

Idea: Recorre cada letra con Subcadena. Si no es ninguna vocal, suma una consonante.

Truco: Usa Y entre cinco comparaciones <>: la condición sólo es cierta si la letra es distinta de TODAS las vocales.

Invertir una palabra (id: invertir_cadena · pasa 10/10 casos)

```
Proceso InvertirPalabra
Definir palabra, inv Como Cadena
Definir i, n Como Entero
Leer palabra
n <- Longitud(palabra)
inv <- ""
Para i <- n Hasta 1 Con Paso -1 Hacer
  inv <- Concatenar(inv, Subcadena(palabra, i, i))
FinPara
Escribir "Invertida: ", inv
FinProceso
```

Idea: Para con paso -1 desde el final hasta el principio. En cada vuelta concatena una letra.

Truco: Inicializa la cadena de resultado a "" (vacío), no a nada.

¿Empieza por vocal? (id: empieza_vocal · pasa 10/10 casos)

```
Proceso EmpiezaVocal
Definir palabra, c Como Cadena
Leer palabra
c <- Subcadena(palabra, 1, 1)
Si c = "a" O c = "e" O c = "i" O c = "o" O c = "u" Entonces
  Escribir "SI"
SiNo
  Escribir "NO"
FinSi
FinProceso
```

Idea: Coge la primera letra con Subcadena(palabra, 1, 1). Cinco comparaciones con O.

Truco: Subcadena en Pselnt es 1-indexada e inclusiva — (s, 1, 1) devuelve el primer carácter.

■ Funciones matemáticas

Área del círculo (id: area_circulo · pasa 10/10 casos)

```
Proceso AreaCirculo
Definir r Como Real
Definir a Como Entero
Leer r
a <- Redon(PI * r ^ 2)
Escribir "Area: ", a
FinProceso
```

Idea: Fórmula clásica $\pi \cdot r^2$. PI es una constante built-in. Redon redondea al entero más cercano.

Truco: Declarar a como Entero hace que se redondee al asignar — pero usamos Redon explícitamente para que se entienda mejor.

Hipotenusa (Pitágoras) (id: hipotenusa · pasa 10/10 casos)

```
Proceso Hipotenusa
Definir a, b Como Real
Definir h Como Entero
Leer a
Leer b
h <- Redon(RC(a ^ 2 + b ^ 2))
Escribir "Hipotenusa: ", h
FinProceso
```

Idea: Teorema de Pitágoras: $h = \sqrt{a^2 + b^2}$. RC es la raíz cuadrada.

Truco: Los test cases usan ternas pitagóricas perfectas (3-4-5, 5-12-13, ...) — para esos la raíz da entero exacto.

Distancia entre dos puntos (id: distancia puntos · pasa 10/10 casos)

```
Proceso Distancia
Definir x1, y1, x2, y2 Como Real
Definir d Como Entero
Leer x1
Leer y1
Leer x2
Leer y2
d <- Redon(RC((x2 - x1) ^ 2 + (y2 - y1) ^ 2))
Escribir "Distancia: ", d
FinProceso
```

Idea: Distancia euclidiana = $\sqrt{(\Delta x)^2 + (\Delta y)^2}$. Es una hipotenusa donde los catetos son las diferencias en X e Y.